

Exercise 6

Ziyu Ma

Environment for running the code:

e.g. when running with Anaconda:

```
conda install pytorch torchvision torchaudio tensorboard pytorch-cuda=11.8 -c pytorch -c nvidia
```

then

```
pip install pytorch-msssim
```

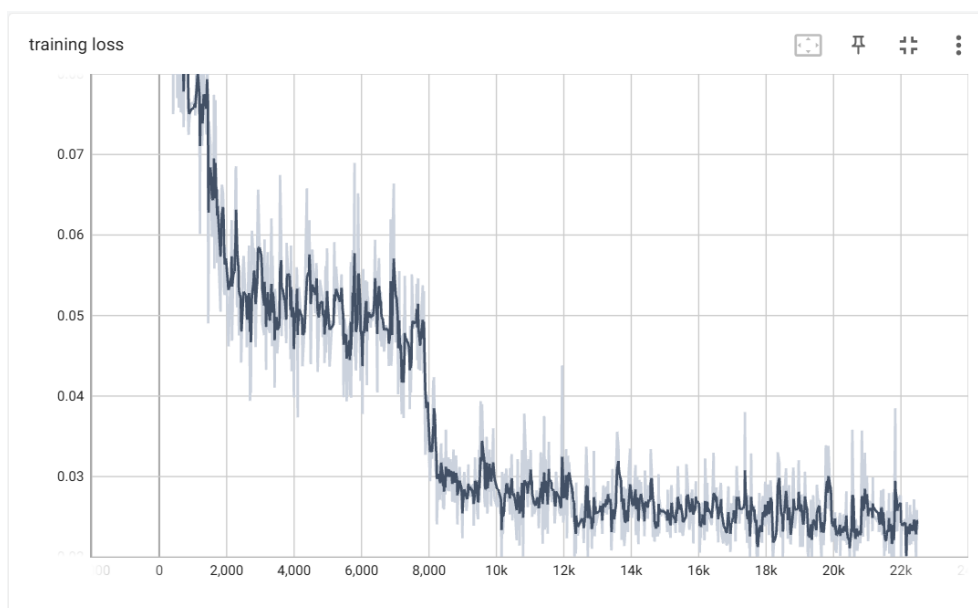
Task 4-Model Evaluation:

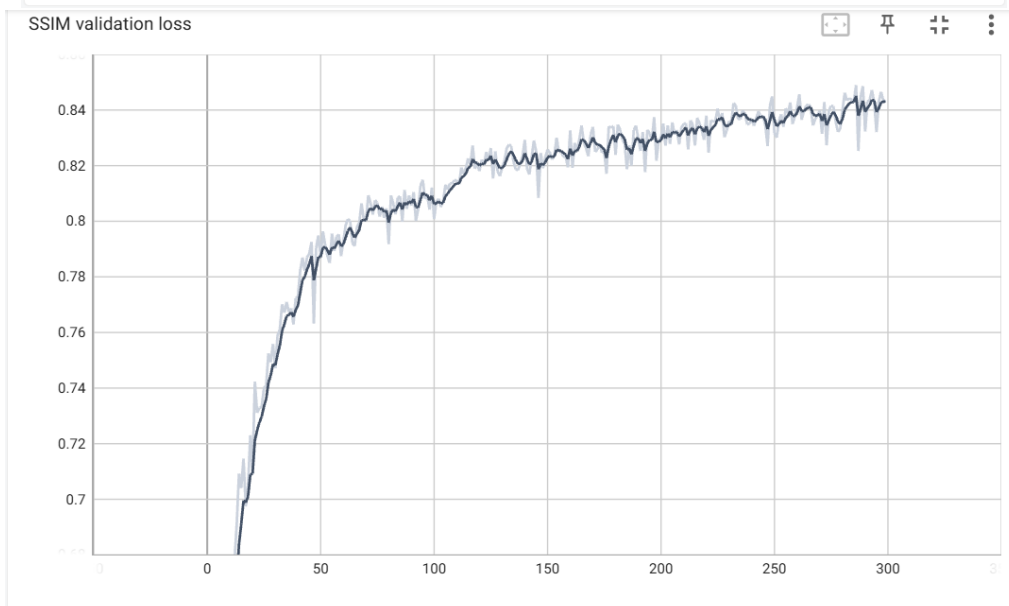
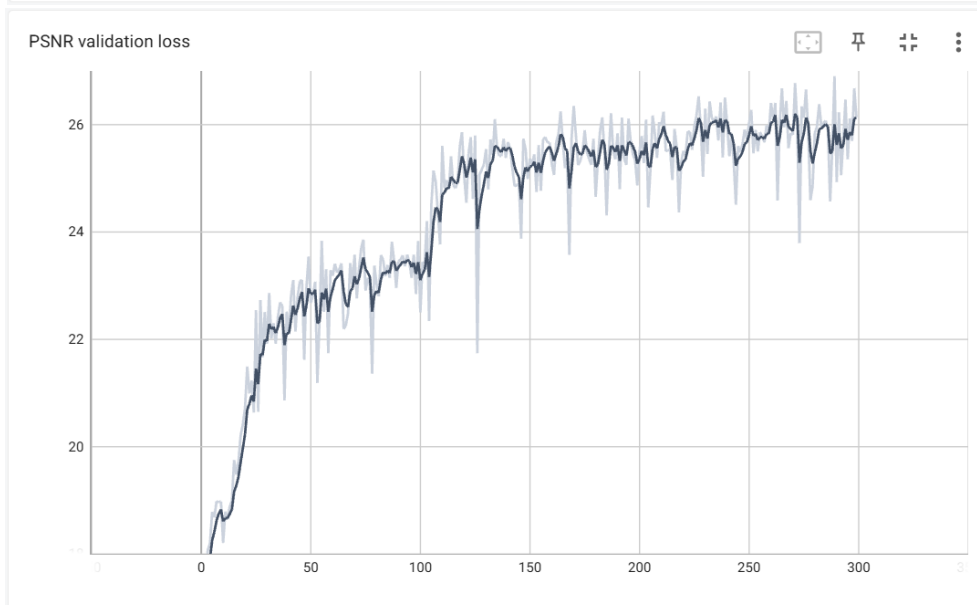
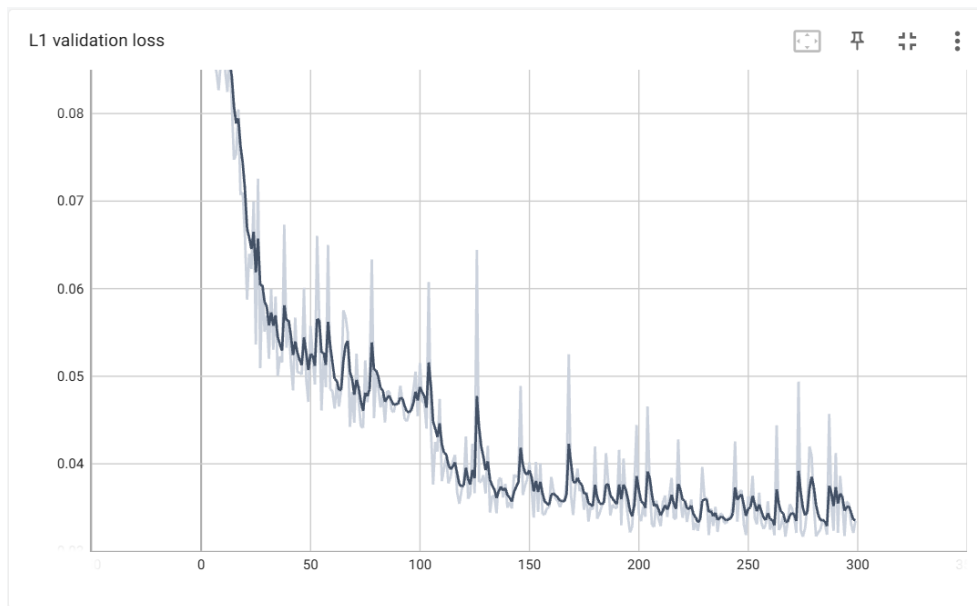
All relevant results (diagrams, checkpoints used for evaluation, result images and events.out for tensorboard can be found in the folder “results/Basic_04”).

The result images of traditional bilinear method can be found in the folder “bilinear_task4” and the ones of traditional bicubic method can be found in the folder “bicubic_task4”.

Result of BasicSRModel with $lr = 1e-4$ (the condition before task 5)

	PSNR loss	SSIM loss
Traditional bilinear	23.5392	0.7442
Traditional bicubic	24.4148	0.7880
BasicSRModel	25.8450	0.8486

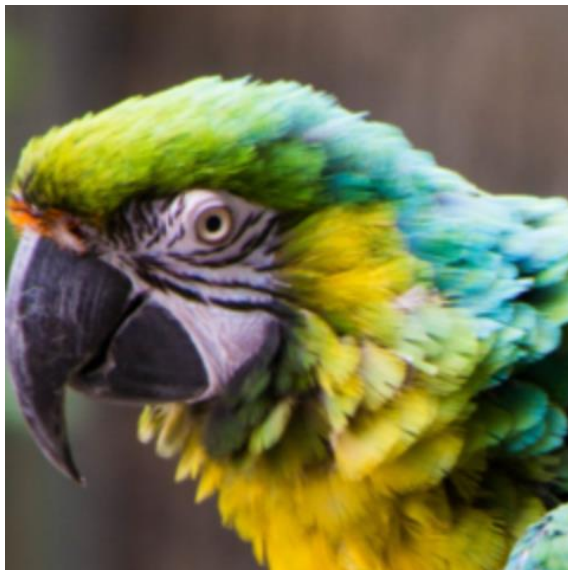




Both PSNR and SSIM loss of BasicSRModel is better than the traditional methods. The result images are also sharper.



bicubic



bilinear



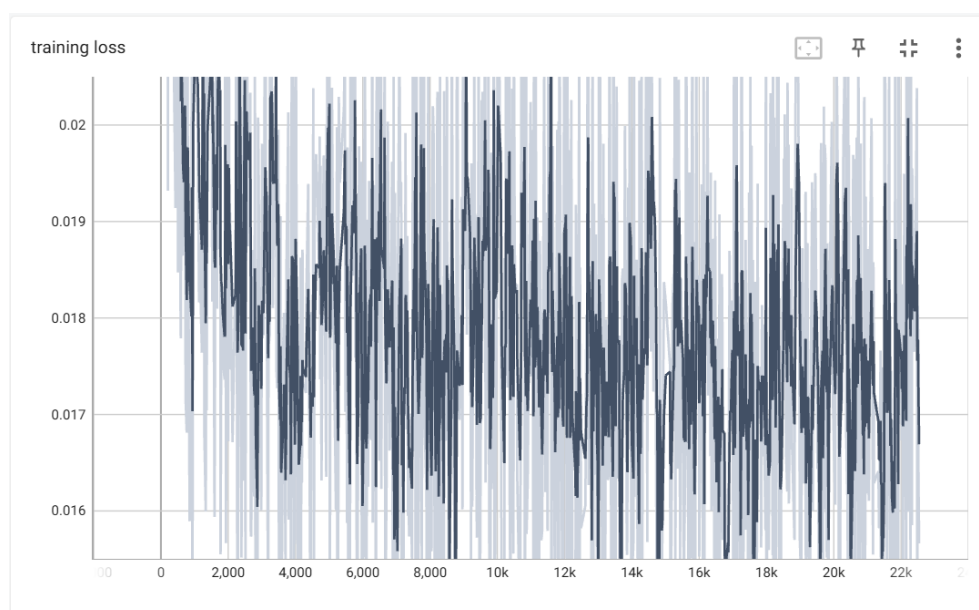
BasicSRModel

Task 5-Exploration

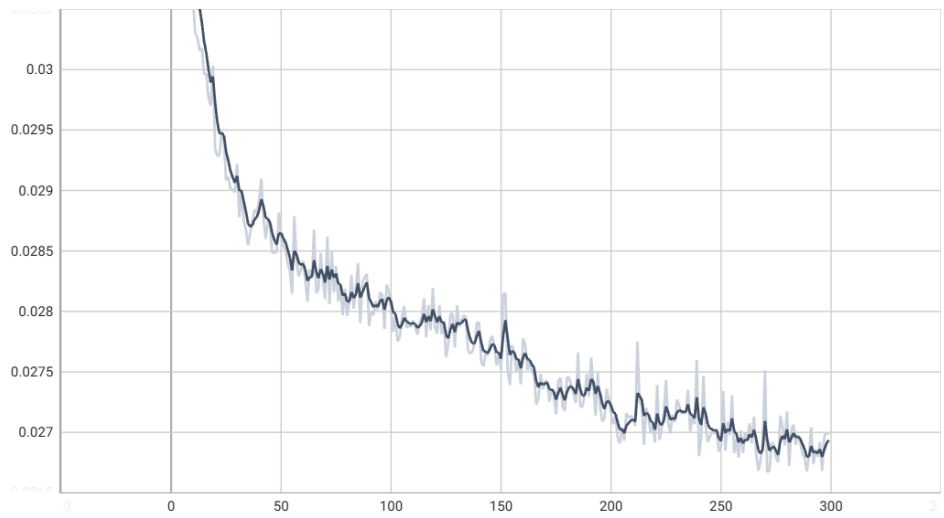
A Different Model

All relevant results (diagrams, checkpoints used for evaluation, result images and events.out... for tensorboard can be found in the folder “results/Residual_04”).

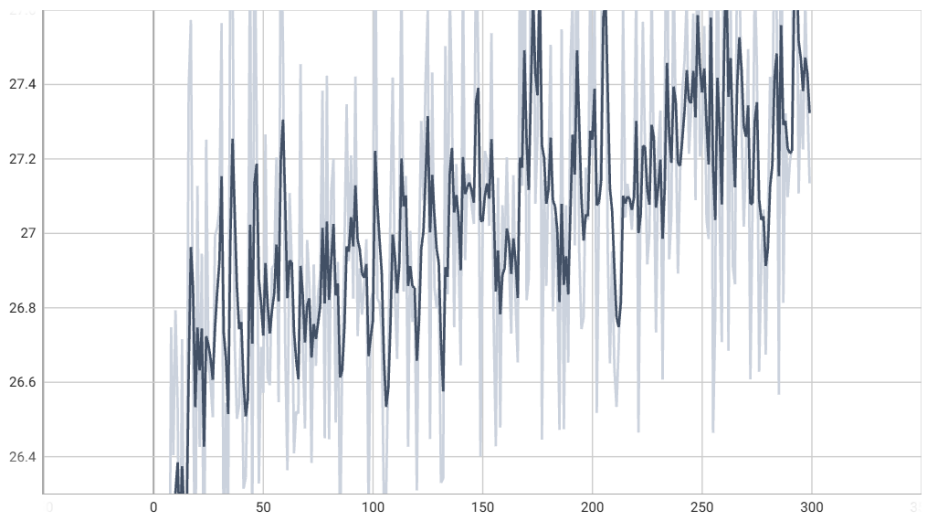
	PSNR loss	SSIM loss
Traditional bilinear	23.5392	0.7442
Traditional bicubic	24.4148	0.7880
BasicSRModel	25.8450	0.8486
ResidualModel	27.9382	0.8714



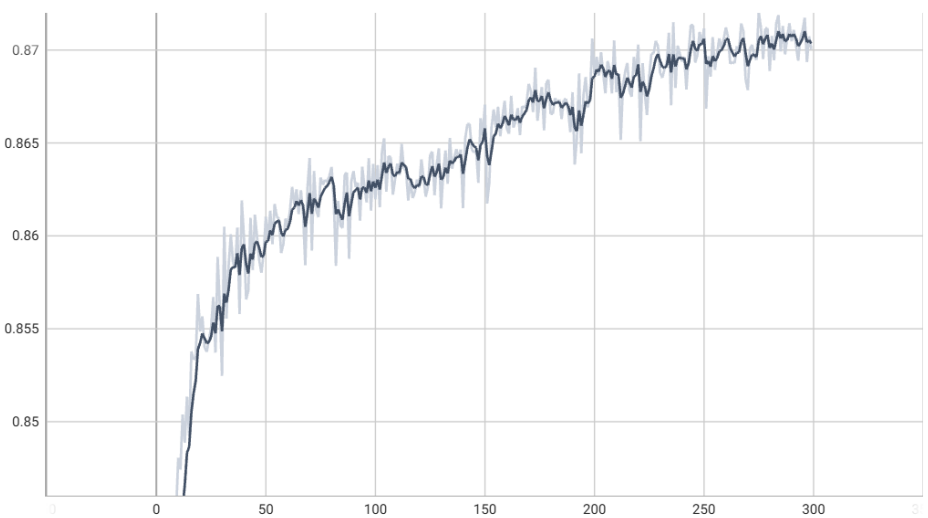
L1 validation loss



PSNR validation loss



SSIM validation loss



Compared to BasicSRModel, the training loss of ResidualModel shrinks much quicker and starts to be within 0 and 0.02 after very few steps. The loss are also smaller. This is because the model tries to predict and fit the difference between the upsampled 32×32 images and the 64×64 images, indeed of the 64×64 images themselves.

After training both models for 300 epochs, both PSNR and SSIM results of the ResidualModel are better than the ones of the BasicSRModel. We can also see that in the result images.



BasicSRModel



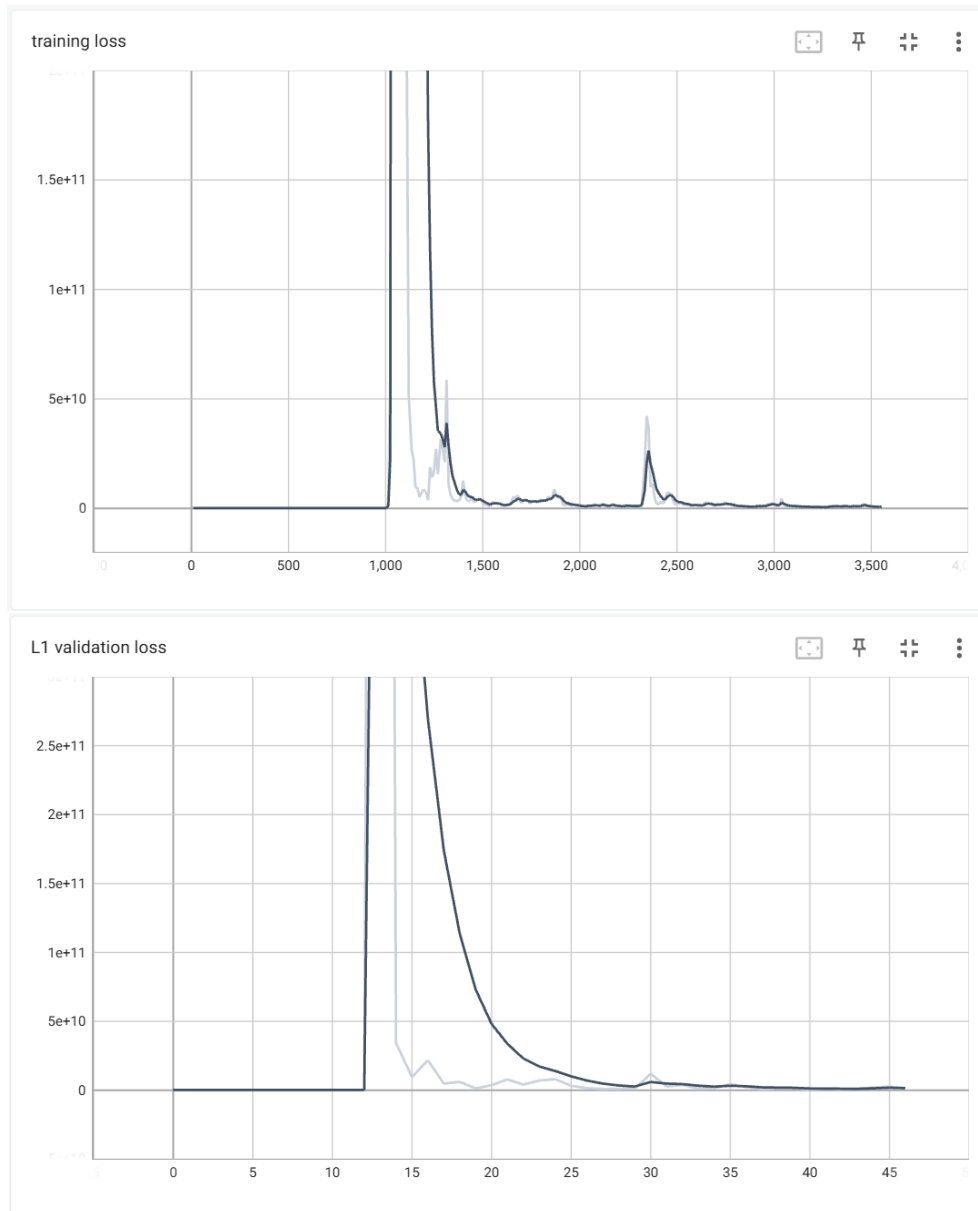
ResidualModel

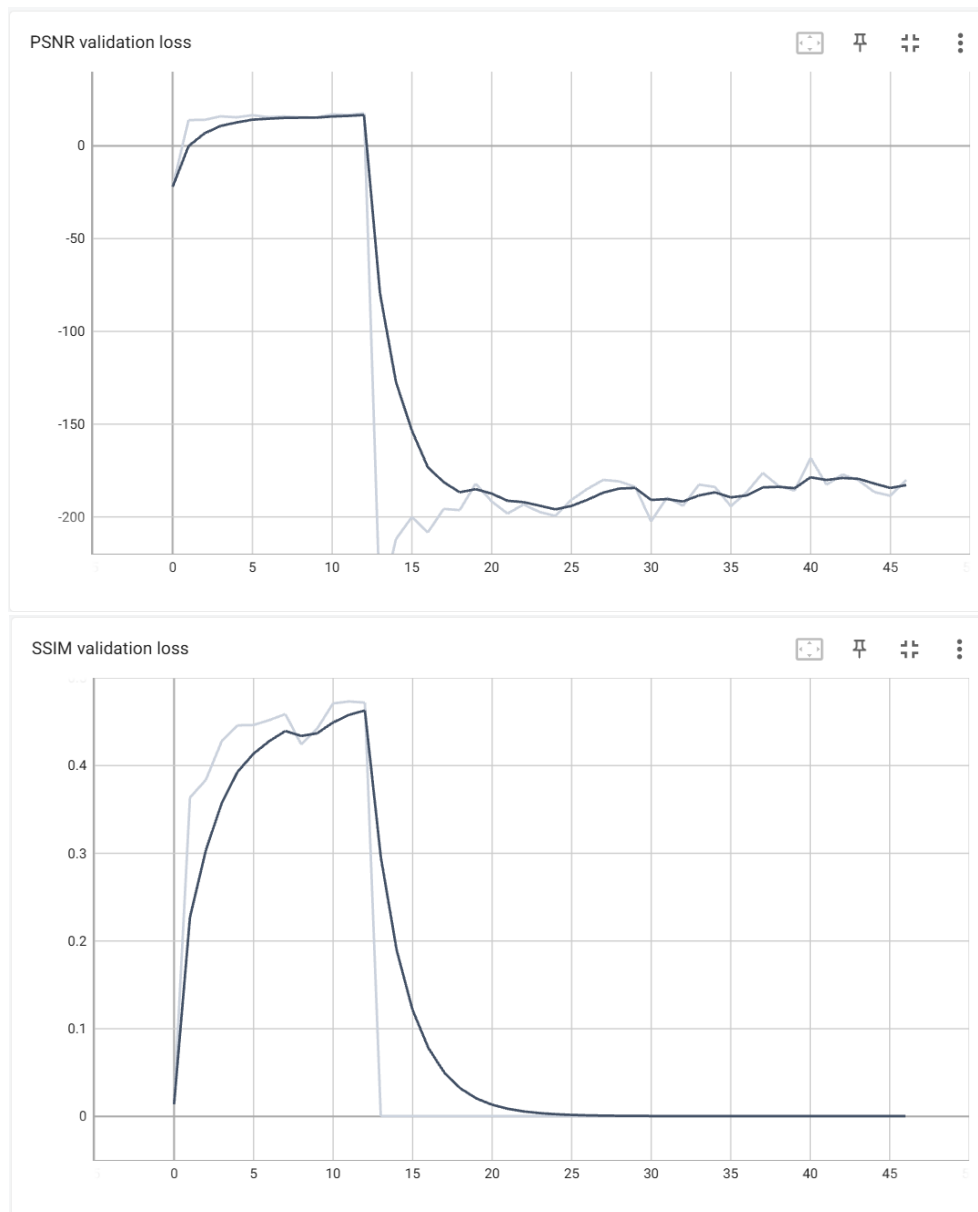
For example, this image of the ResidualModel is sharper than the one of the BasicSRModel.

Effect of the Learning Rate

$lr = 1e-2$

All relevant results (diagrams, checkpoints used for evaluation, result images and events.out... for tensorboard can be found in the folder “results/Basic_02”).





When $lr = 1e-2$, the diagrams start to turn crazy after about 12 epochs. The L1 training and validation losses become really large for a few iterations and then fall down, but still have size $e+8/e+9$ which is really large. The smallest L1 training loss is about 0.10 and the smallest validation L1 loss is about 0.13.

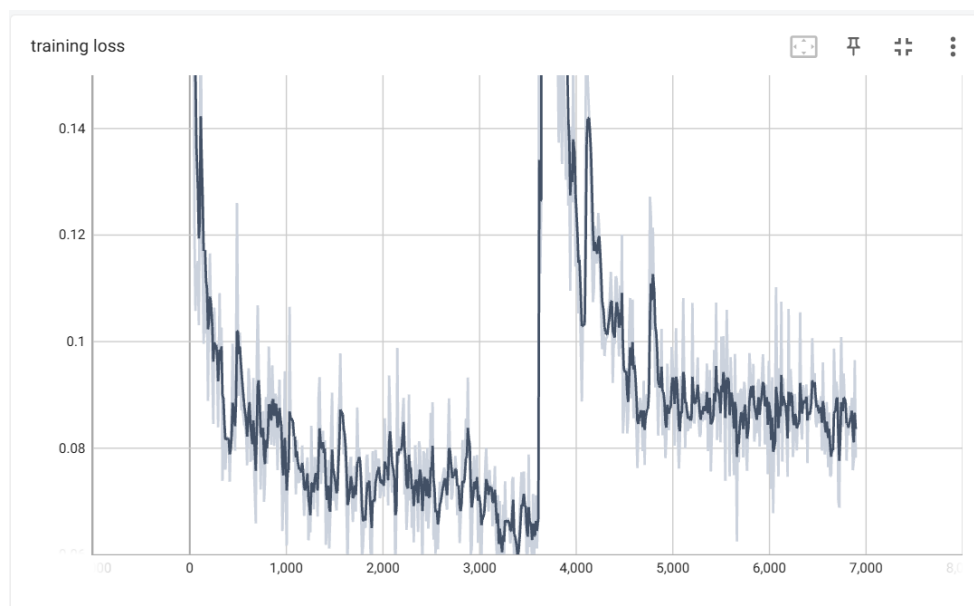
The PSNR and SSIM values go up also before about 12 epochs and then decreases drastically. The highest score of SSIM is about 0.47, while the highest score of PSNR is about 17.5. Both are worse than the traditional methods.

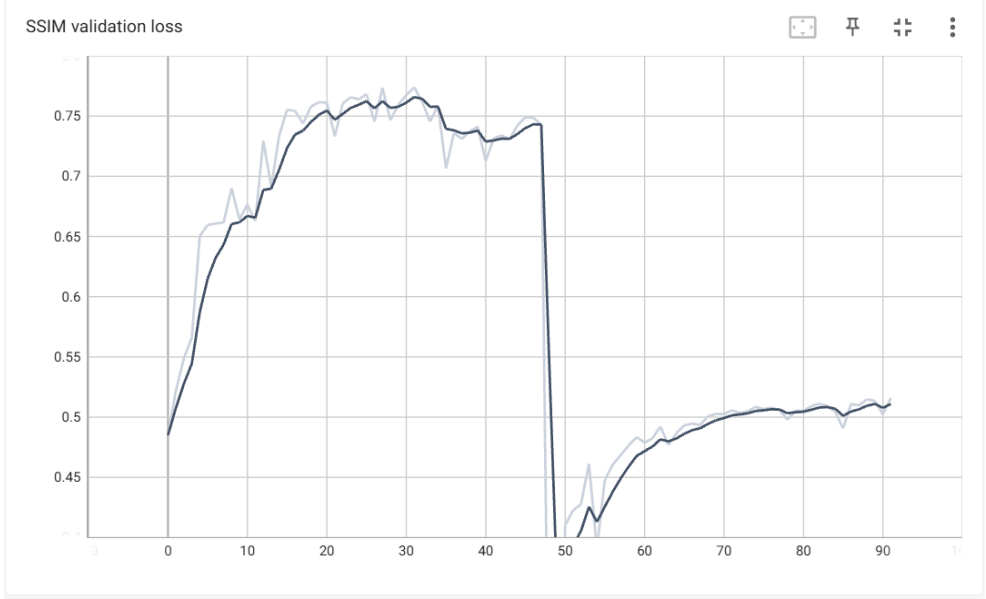
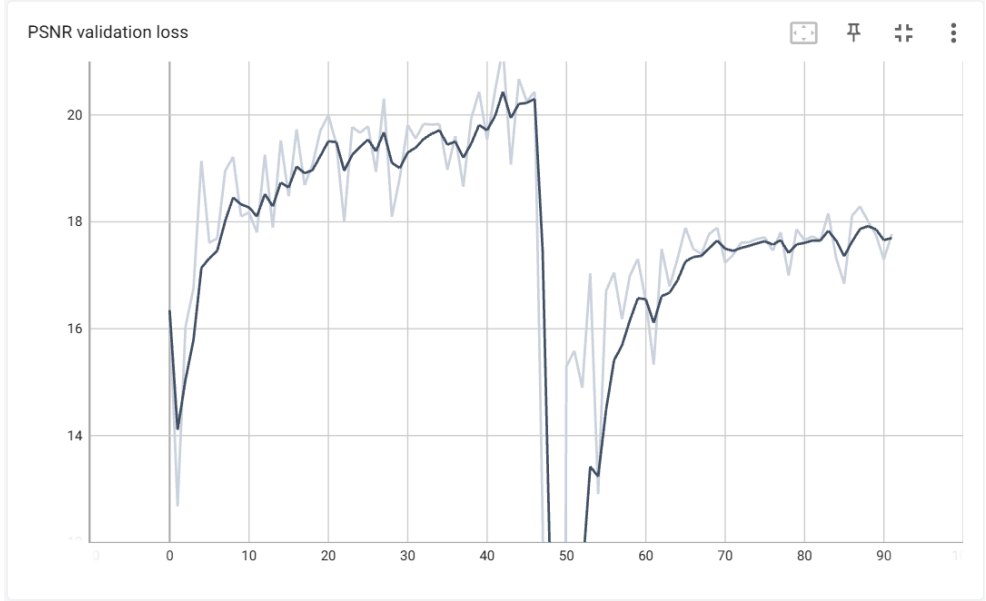
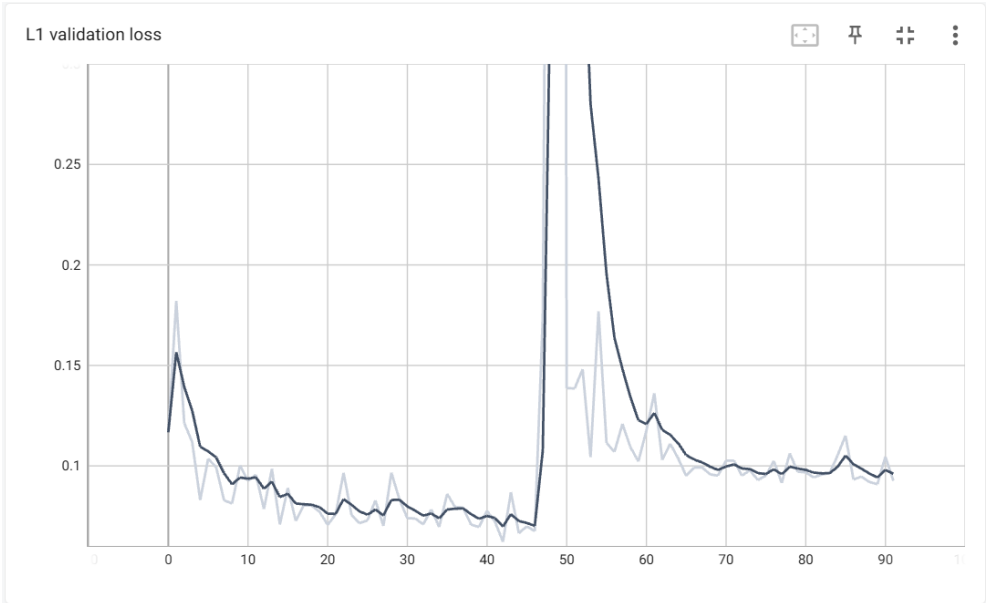
The result images are lack of color.



$lr = 1e-3$

All relevant results (diagrams, checkpoints used for evaluation, result images and events.out... for tensorboard can be found in the folder “results/Basic_03”).





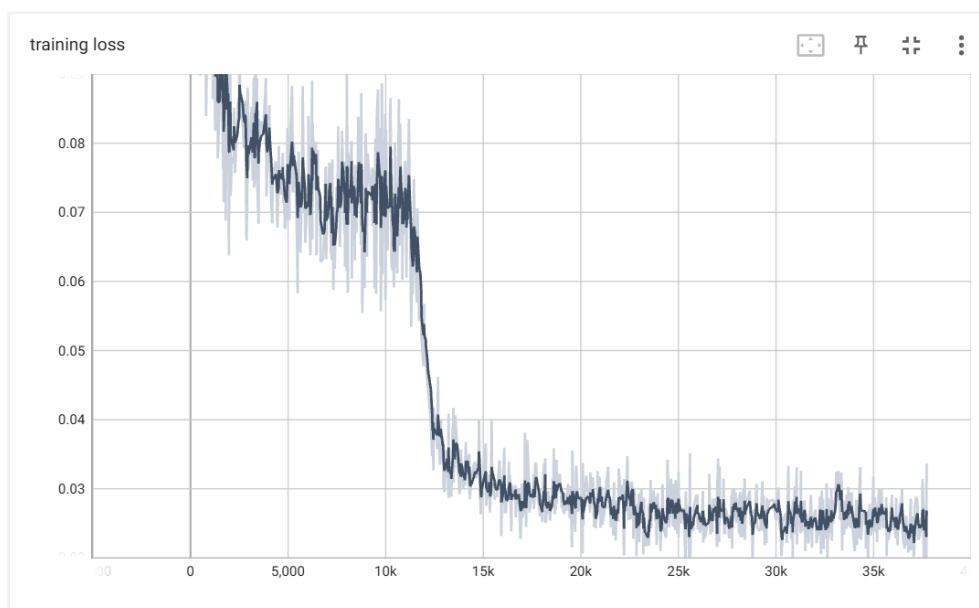
The situation is similar to $lr = 1e-2$, while the diagrams behave well for more epochs and the results are better. The smallest L1 training loss is about 0.05 and the smallest validation L1 loss is about 0.07. The highest score of SSIM is about 0.76, while the highest score of PSNR is about 21.25. Both are still worse than the traditional methods.

The result images are still lack of color, but have more information then $lr = 1e-2$, e.g. more contrast of light and dark.

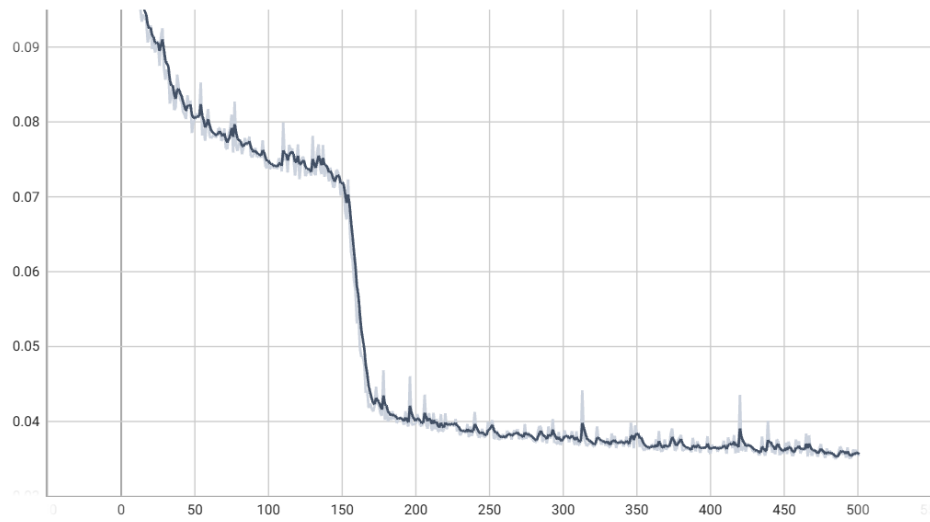


$lr = 1e-5$

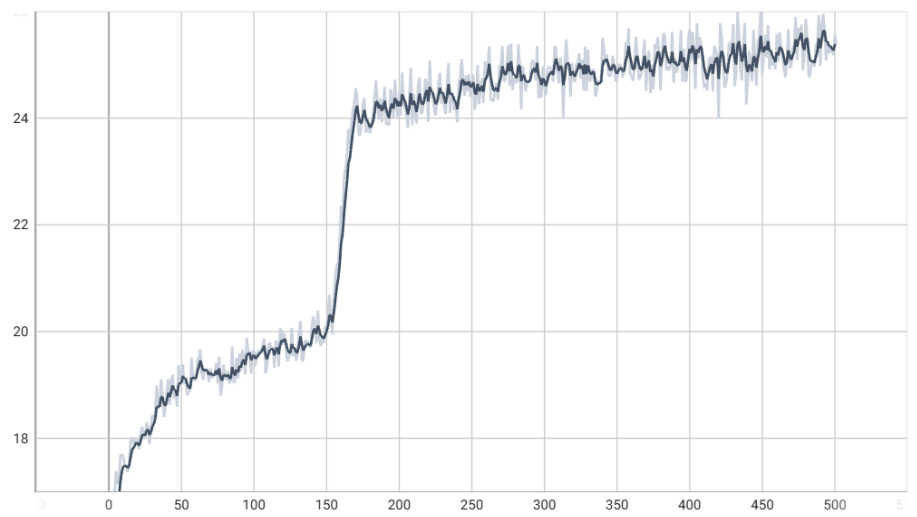
All relevant results (diagrams, checkpoints used for evaluation, result images and events.out... for tensorboard can be found in the folder “results/Basic_05”).



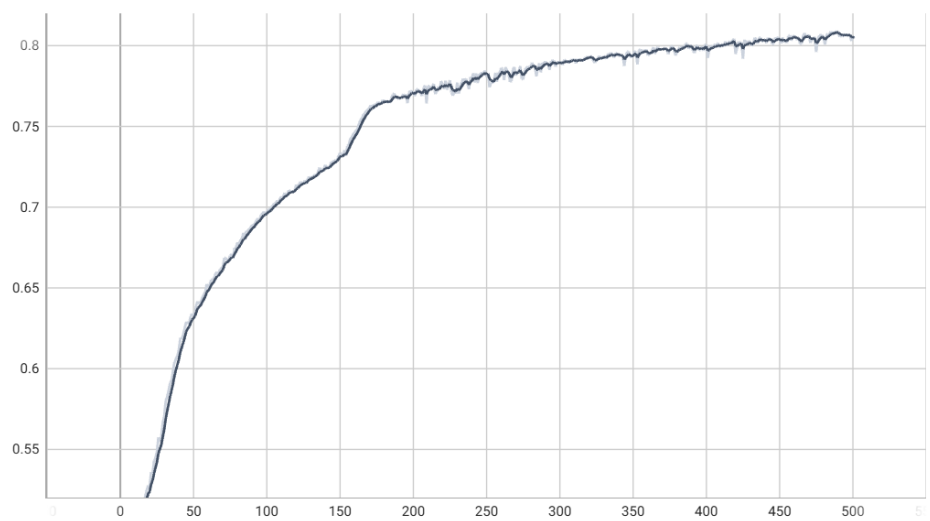
L1 validation loss



PSNR validation loss



SSIM validation loss



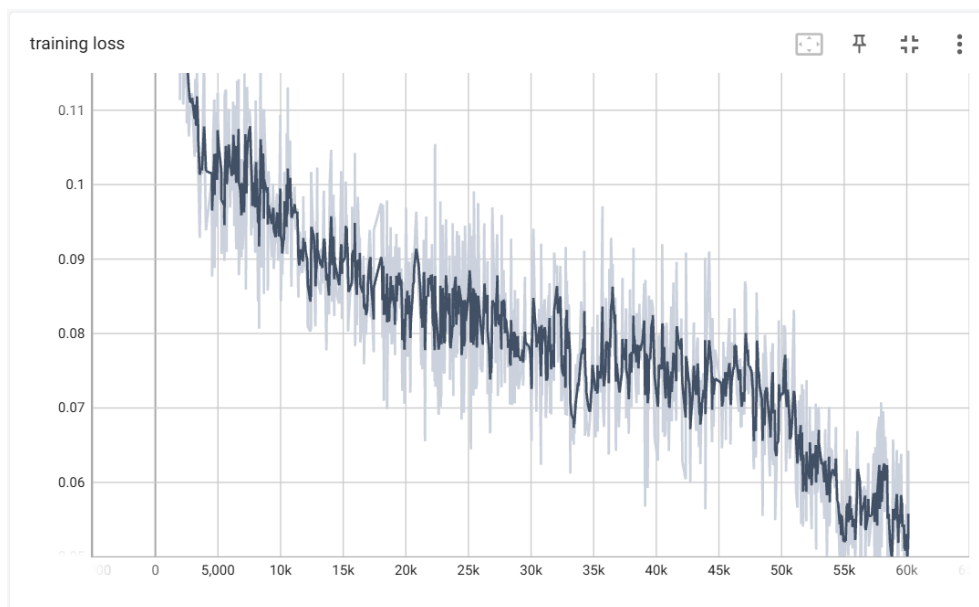
I trained the model for 500 epochs. The model seems to get in a local minimum easily and gets out of it at about 170 epochs. The model is lack of efficiency compared to $lr = 1e-4$. Even though it is better than the traditional ones, it is worse than the BasicSRModel ($lr = 1e-4$) even after much more training epochs.

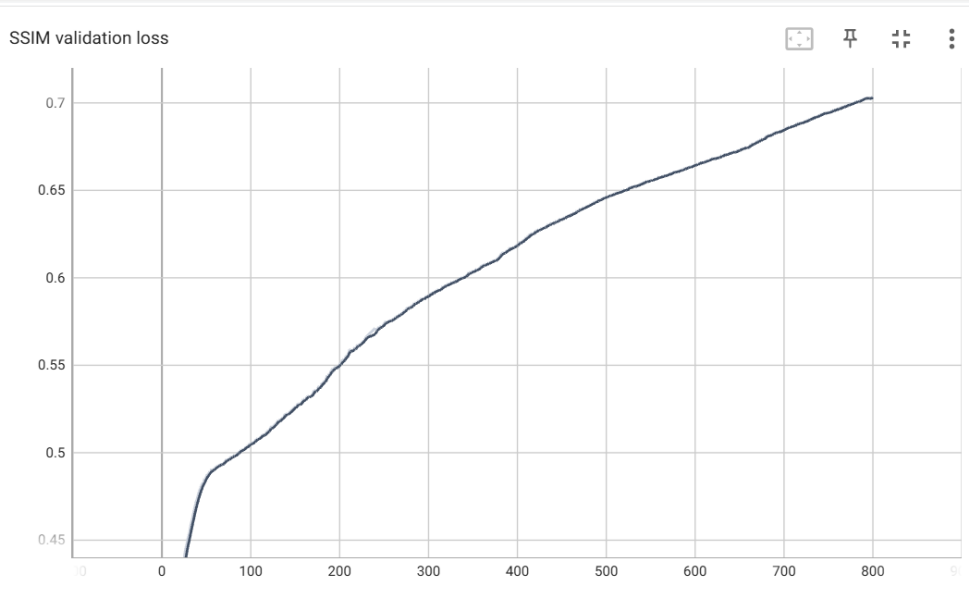
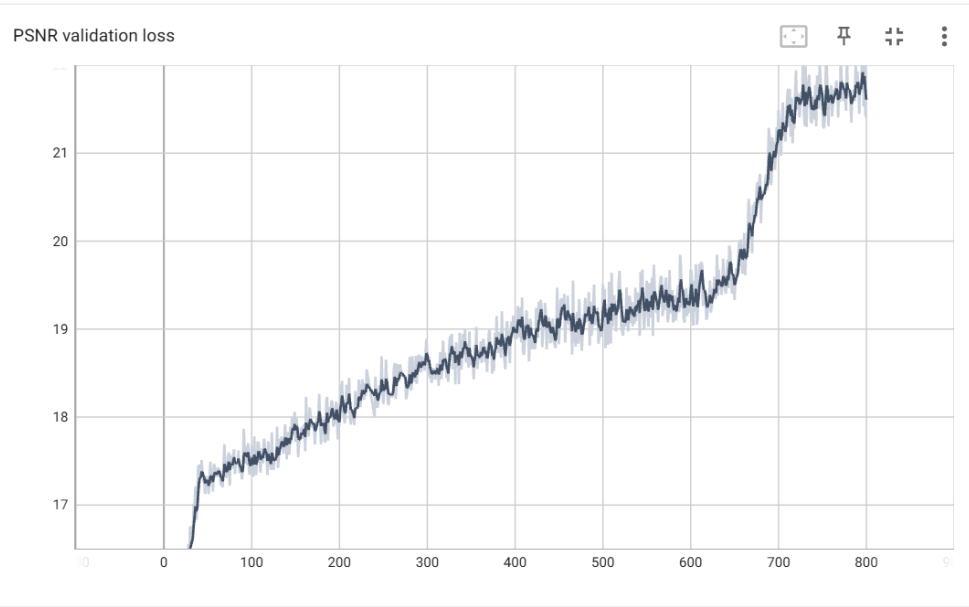
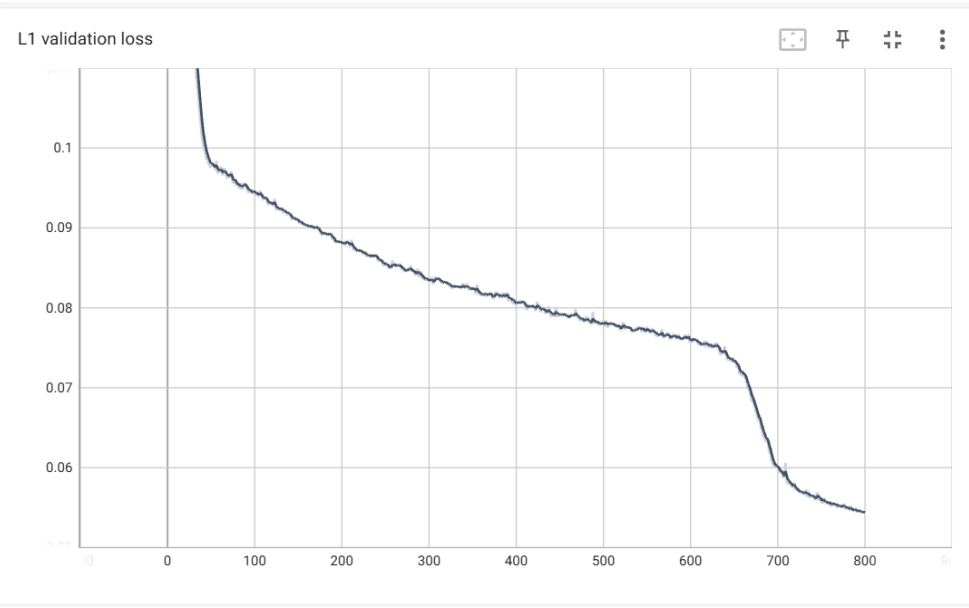
The images have colors.



$lr = 1e-6$

All relevant results (diagrams, checkpoints used for evaluation, result images and events.out... for tensorboard can be found in the folder “results/Basic_06”).





The efficiency is worse than the one with $lr = 1e-5$. Even after 800 epochs of training, the PSNR and SSIM loss are still worse than the traditional ones. Even though the images have colors, the colors differ a lot with the original ones.



Here is the table after adding all the models with different learning rates.

	PSNR loss	SSIM loss
Traditional bilinear	23.5392	0.7442
Traditional bicubic	24.4148	0.7880
BasicSRModel ($lr = 1e-4$)	25.8450	0.8486
ResidualModel	27.9382	0.8714
BasicSRModel ($lr = 1e-2$)	15.2422	0.4421
BasicSRModel ($lr = 1e-3$)	20.3544	0.7411
BasicSRModel ($lr = 1e-5$, 500 epochs)	25.7177	0.8070
BasicSRModel ($lr = 1e-6$, 800 epochs)	21.8313	0.7032

Different Downscaling During Inference

All result images can be found in the folder
“results/Different_Downscaling_Bicubic” and
“results/Different_Downscaling_Nearest”.

Evaluation	PSNR loss	SSIM loss
Downscaling bicubic	25.2314	0.8423
Downscaling Nearest	18.7205	0.6264

I use the model BasicSRModel ($lr = 1e-4$) to evaluate the two different downsampling methods. The bicubic one is slightly worse than the bilinear downsampling, while the nearest neighbor one is much worse. The reason is the the model is trained to upscale bilinearly downsampled images (we have bilinear downsample in the training dataset, then bilinear upsample in the model).

The result images have almost no difference in color, but we can see particles in e.g. sharp edges. This is usually caused by wrong upsampling method. E.g. in the following

image with Nearest neighbor downsampling.

